

Yesterday 11:12 AM

i will give you the java question you have to answer it according to 5 yr of exp in jav

Absolutely. 👍

Give me any **Java interview question**, and I'll answer it at the level expected from a **5-year experienced Java/Spring Boot developer**, especially for **product-based companies like Amazon, Netflix, Google, Microsoft, etc.**

For each question, I'll structure the answer like this:

1. **Interview-ready answer** — what you should actually say to the interviewer.
2. **Concept explanation** — easy-to-understand explanation so you genuinely understand it.
3. **Internal working** — JVM / memory / threads / collections internals wherever relevant.
4. **Code example** — practical Java code when applicable.
5. **Real-world usage** — how it is used in a production application.
6. **5-year experience perspective** — what an interviewer expects you to know beyond the basic definition.
7. **Follow-up questions** — likely questions the interviewer may ask next, with answers.
8. **Common mistakes** — things you should avoid saying in an interview.

I'll also help you **represent the answer properly in an interview**, rather than just giving a textbook definition.

For example, instead of:

"HashMap stores key-value pairs and uses hashing."

I would help you answer more like:

"HashMap is a non-thread-safe key-value data structure. Internally, it maintains an array of buckets, and the bucket is determined using the key's hash. In modern Java, when multiple keys fall into the same bucket, entries are initially maintained as a linked structure, and under certain conditions the bucket can be converted into a balanced

tree to improve lookup performance. Average `get` and `put` are $O(1)$, although collisions can affect this..."

Then I can go deeper into **hash calculation** → **bucket index** → **collision** → **treeification** → **resizing** → **load factor** → **Java 8+ implementation**, depending on what the interviewer asks.

Send me your first Java question.



Why is Java pass-by-value even when we pass an object?

This is a **very common 5-year Java interview question**. The key is to clearly distinguish between **passing an object** and **passing the reference value**.

Interview-ready answer

Java is always pass-by-value. When we pass an object to a method, Java does not pass the actual object or pass the reference itself by reference. Instead, it passes a **copy of the reference value**.

Both the original reference and the copied reference point to the **same object in the heap**. Therefore, if the method modifies the object's fields through that copied reference, the changes are visible to the caller.

However, if we reassign the parameter to a new object, only the local copy of the reference changes. The caller's reference remains unchanged.

So the important distinction is: **Java passes the reference value by value.**

Let's understand it step-by-step

Consider:

</> Java



```

class Employee {
    String name;
}

public class Test {

    static void change(Employee e) {
        e.name = "Rahul";
    }

    public static void main(String[] args) {

        Employee emp = new Employee();
        emp.name = "Manoj";

        change(emp);

        System.out.println(emp.name);
    }
}

```

Output:

Rahul



You might think:

"The object was passed by reference."

That's not technically correct in Java.

What actually happens?

Initially:

```

main()

emp
|
| reference value
↓
+-----+
| Employee |
| name = Manoj |
+-----+
      Heap

```



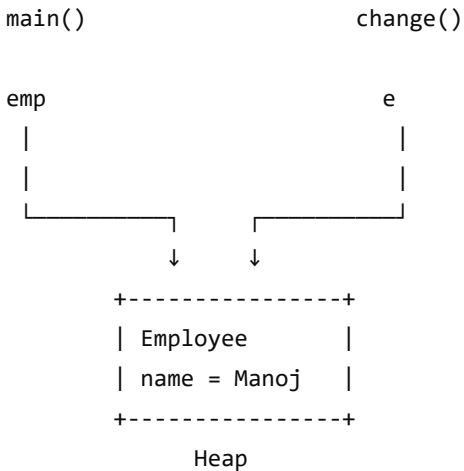
When you call:

```
</> Java
```

```
change(emp);
```

Java copies the **reference value** stored in `emp` .

Now:



There are **two reference variables**:

```
emp  
e
```

But both contain a reference to the **same Employee object**.

Therefore:

```
</> Java
```

```
e.name = "Rahul";
```

modifies the same object.

So `emp.name` also becomes `"Rahul"` .

The important proof: reassignment

Now consider this:

</> **Java**

```
static void change(Employee e) {  
  
    e = new Employee();  
    e.name = "Rahul";  
}
```

And:

</> **Java**

```
Employee emp = new Employee();  
emp.name = "Manoj";  
  
change(emp);  
  
System.out.println(emp.name);
```

Output:

Manoj



Why?

Before method call:

main()



emp

|

↓

Object A

name = Manoj

During method call:

main()

change()



emp

e

|

|

Object A
name=Manoj

Remember:

`e` is a **copy of the reference value**.

Now:

</> **Java**

```
e = new Employee();
```



We change only the local variable `e`.

main()

change()



emp

e

|

|

↓

↓

Object A

Object B

Manoj

Rahul

The caller's `emp` still points to **Object A**.

Therefore:

</> **Java**

```
System.out.println(emp.name);
```



prints:

Manoj



The easiest way to remember it

Think of the reference as an **address/value** that tells Java where the object is.

Suppose:

</> Java



```
Employee emp = new Employee();
```

Conceptually:

emp = 0x100



And:

0x100 → Employee object



When you call:

</> Java



```
change(emp);
```

Java does:

e = 0x100



So:

emp = 0x100

e = 0x100



Both point to the same object.

But they are **two separate variables**.

If you do:

</> Java



```
e.name = "Rahul";
```

you're modifying the object at 0x100 .

But if you do:

</> Java



```
e = 0x200;
```

you're only changing the local variable `e` .

`emp` is still:

```
emp = 0x100
```



That's why Java is **pass-by-value**.

Primitive vs Object

This comparison is excellent to give in an interview:

Passing	What gets copied?	Can method change caller's variable?
<code>int</code>	Value of <code>int</code>	✗ No
<code>double</code>	Value of <code>double</code>	✗ No
<code>Employee</code> object	Value of the reference	✗ Cannot reassign caller's reference
Object fields	Same underlying object	✓ Can modify fields

Example with primitive:

```
</> Java
static void change(int x) {
    x = 20;
}

int a = 10;

change(a);

System.out.println(a);
```



Output:

```
10
```



Because:

```
a = 10
```



```
x = 10 // copy
```

Then:

```
x = 20
```



doesn't affect a .

With an object:

```
</> Java
```



```
static void change(Employee e) {  
    e.name = "Rahul";  
}
```

e is still a copied value, but that copied value points to the same object.

A very important interview statement

If the interviewer asks:

"So does Java pass objects by reference?"

Don't say:

Yes, objects are passed by reference.

Instead say:

No. Java is strictly pass-by-value. For objects, the value being passed is the reference to the object. Therefore, a copy of the reference is passed. Both references can access the same object, which is why mutations to the object's state are visible to the caller, but reassignment of the parameter does not affect the caller's reference.

That's the 5-year-experience-level answer.

One-line memory trick

Java doesn't pass the object by reference; Java passes the object's reference value by value.



Collection Selection — Scenario-Based Interview Notes

1. You need an LRU cache. Which collection?

Choose: `LinkedHashMap`

Use **access-order** mode:

`</>` Java



```
new LinkedHashMap<>(16, 0.75f, true);
```

Why?

- Average $O(1)$ lookup
- Maintains access order
- Recently accessed entry moves to the end
- `removeEldestEntry()` can remove the least recently used entry

LRU → B → C → A ← Most recently used



Interview answer: `LinkedHashMap` with access-order is a simple way to implement an LRU cache.

2. 90% reads + 10% writes on a shared List

Choose: `CopyOnWriteArrayList`

Why?

Reads → very frequent
Writes → relatively less frequent



`CopyOnWriteArrayList` allows efficient read access while writes create a new underlying array.

Best suited when:

Reads >>> Writes



Examples:

- Listener lists
- Configuration data
- Subscriber lists

Interview answer: Since the workload is read-heavy, I'd choose `CopyOnWriteArrayList`, accepting the higher cost of writes for efficient concurrent reads.

3. Multiple threads frequently reading and modifying a Map

Choose: `ConcurrentHashMap`

</> Java



```
Map<String, User> users =  
    new ConcurrentHashMap<>();
```

Why?

- Thread-safe
- Designed for concurrent access
- Allows concurrent reads and updates
- Better scalability than synchronizing the entire Map

Interview answer: I'd use `ConcurrentHashMap` for a shared Map with frequent concurrent reads and modifications.

4. Queue for BFS

Choose: ArrayDeque

</> Java



```
Queue<Node> queue = new ArrayDeque<>();
```

BFS requires FIFO:

First added → First processed



Operations:

</> Java



```
queue.offer(node);  
queue.poll();
```

Both are efficient.

Why not LinkedList?

LinkedList also works as a Queue, but ArrayDeque is generally preferred because it has:

- Better cache locality
- Lower memory overhead
- No per-node object allocation

Interview answer: For BFS, I'd normally use ArrayDeque as the queue implementation.

5. Priority-based task scheduler

Choose: PriorityQueue

</> Java



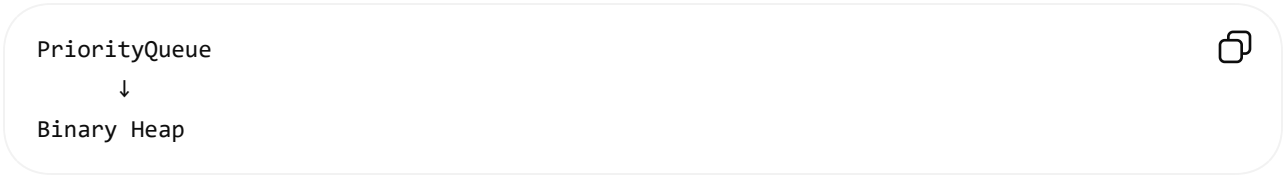
```
PriorityQueue<Task> queue =  
    new PriorityQueue<>(  
        Comparator.comparing(Task::getPriority)  
    );
```

The highest-priority task can be retrieved using:

</> Java

queue.peek();
queue.poll();

Internally:



Complexity:

peek() → O(1)
offer() → O(log n)
poll() → O(log n)

If multiple threads are producing/consuming tasks, consider:

</> Java

PriorityBlockingQueue

instead.

Interview answer: For a single-threaded priority queue, I'd use `PriorityQueue` . For a concurrent producer-consumer scheduler, I'd consider `PriorityBlockingQueue` .

★ Quick Interview Table

Scenario	Choice	Why
LRU Cache	LinkedHashMap	Access order + O(1) average lookup
90% reads / 10% writes List	CopyOnWriteArrayList	Read-heavy workload
Concurrent Map	ConcurrentHashMap	Scalable concurrent access
BFS Queue	ArrayDeque	Efficient FIFO

Scenario	Choice	Why
Priority scheduler	PriorityQueue	Binary heap + priority ordering
Concurrent priority scheduler	PriorityBlockingQueue	Thread-safe priority queue

5-Year Shortcut

LRU → LinkedHashMap
 Read-heavy List → CopyOnWriteArrayList
 Concurrent Map → ConcurrentHashMap
 BFS → ArrayDeque
 Priority processing → PriorityQueue
 Concurrent priority → PriorityBlockingQueue



You need a queue for BFS. Which implementation would you choose?

You need a priority-based task scheduler. Which collection would you choose?

You need to remove duplicates while preserving order.

How would you do it?

How would you find the first non-repeating character using Java Collections?

How would you find duplicate elements in an array using Collections?

How would you count frequency of each element using a Map?

How would you sort a Map by values?

How would you find the top K frequent elements?

Java Collections — Coding-Based Interview Notes

1. Queue for BFS — which implementation?

Choose `ArrayDeque` .

</> Java



```
Queue<Node> queue = new ArrayDeque<>();

queue.offer(root);

while (!queue.isEmpty()) {
    Node node = queue.poll();
    // process node
}
```

Why: BFS needs FIFO, and `ArrayDeque` provides efficient `offer()` / `poll()` with lower overhead than `LinkedList`.

2. Priority-based task scheduler — which collection?

Choose `PriorityQueue`.

</> Java



```
PriorityQueue<Task> queue =
    new PriorityQueue<>(
        Comparator.comparing(Task::getPriority)
    );
```

```
peek() → O(1)
offer() → O(log n)
poll() → O(log n)
```



For multiple producer/consumer threads, use `PriorityBlockingQueue`.

3. Remove duplicates while preserving order

Use `LinkedHashSet`.

</> Java



```
List<Integer> list =
    Arrays.asList(10, 20, 10, 30, 20);
```

```
List<Integer> result =  
    new ArrayList<>(new LinkedHashSet<>(list));
```

Result:

[10, 20, 30]



Why: `LinkedHashSet` gives both uniqueness + insertion order.

4. Find the first non-repeating character

Use `LinkedHashMap` to maintain character insertion order and frequency.

`</>` Java



```
String str = "swiss";  
  
Map<Character, Integer> frequency =  
    new LinkedHashMap<>();  
  
for (char ch : str.toCharArray()) {  
    frequency.merge(ch, 1, Integer::sum);  
}  
  
for (Map.Entry<Character, Integer> entry :  
    frequency.entrySet()) {  
  
    if (entry.getValue() == 1) {  
        System.out.println(entry.getKey());  
        break;  
    }  
}
```

Output:

w



Why `LinkedHashMap` ?

HashMap
→ frequency



LinkedHashMap

→ frequency + original order

Time: $O(n)$

Space: $O(k)$, where k is the number of distinct characters.

5. Find duplicate elements in an array

A clean approach is using a `HashSet`.

</> Java



```
int[] arr = {1, 2, 3, 2, 4, 1, 5};

Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = new HashSet<>();

for (int num : arr) {
    if (!seen.add(num)) {
        duplicates.add(num);
    }
}

System.out.println(duplicates);
```

Output:

[1, 2]



Important:

</> Java



```
seen.add(num)
```

returns:

```
true → element was new
false → element already existed
```



Time: $O(n)$ average

Space: $O(n)$

6. Count frequency of each element using a Map

Use `HashMap` .

`</>` Java



```
int[] arr = {1, 2, 2, 3, 3, 3};

Map<Integer, Integer> frequency =
    new HashMap<>();

for (int num : arr) {
    frequency.merge(num, 1, Integer::sum);
}
```

Result:

```
1 → 1
2 → 2
3 → 3
```



Alternative:

`</>` Java



```
frequency.put(
    num,
    frequency.getDefault(num, 0) + 1
);
```

Interview preference

`merge()` is concise and clearly expresses the frequency-counting operation.

7. Sort a Map by values

A Map itself isn't inherently sorted by values, so commonly we sort its entries.

`</>` Java



```
Map<String, Integer> map = new HashMap<>();
```

```
map.put("A", 30);
map.put("B", 10);
map.put("C", 20);

Map<String, Integer> sorted =
    map.entrySet()
        .stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(
            Map.Entry::getKey,
            Map.Entry::getValue,
            (a, b) -> a,
            LinkedHashMap::new
        ));
```

Result:

```
B → 10
C → 20
A → 30
```



Why LinkedHashMap at the end?

Because after sorting the stream, we need a Map implementation that **preserves the sorted iteration order**.

For descending order:

```
</> Java

.sorted(
    Map.Entry.<String, Integer>
        comparingByValue()
        .reversed()
)
```



8. Find the Top K frequent elements

Use:

```
HashMap + PriorityQueue
```



Step 1 — Count frequency

</> Java



```
Map<Integer, Integer> frequency =
    new HashMap<>();

for (int num : arr) {
    frequency.merge(num, 1, Integer::sum);
}
```

Step 2 — Maintain a min-heap of size K

</> Java



```
PriorityQueue<Map.Entry<Integer, Integer>> pq =
    new PriorityQueue<>(
        Map.Entry.comparingByValue()
    );

for (Map.Entry<Integer, Integer> entry :
    frequency.entrySet()) {

    pq.offer(entry);

    if (pq.size() > k) {
        pq.poll();
    }
}
```

At the end:

PriorityQueue



contains K most frequent elements



Why a min-heap?

We want to remove the **least frequent** among the current top K.

Top K candidates



min-heap



least frequent → root

↓

remove when size > K

Complexity

For n array elements and m distinct elements:

Frequency counting $\rightarrow O(n)$
Heap processing $\rightarrow O(m \log K)$

Total $\rightarrow O(n + m \log K)$

Space:

$O(m + K)$

★ 5-Year Interview Summary

Problem	Best approach
BFS	ArrayDeque
Priority tasks	PriorityQueue
Remove duplicates + preserve order	LinkedHashSet
First non-repeating character	LinkedHashMap
Find duplicates	HashSet
Frequency counting	HashMap
Sort Map by values	Stream + LinkedHashMap
Top K frequent	HashMap + PriorityQueue

🎯 Most important patterns to remember

Unique + order
 $\rightarrow$ LinkedHashSet

Frequency

→ HashMap

Frequency + order

→ LinkedHashMap

Top K

→ HashMap + Heap

BFS

→ Queue + ArrayDeque

Priority

→ PriorityQueue

These are very common **5-year Java interview coding patterns** because they test whether you can choose the appropriate collection rather than just knowing the API.



How would you implement an LRU cache?

How would you design a thread-safe in-memory cache?

1. How would you implement an LRU Cache?

For a simple LRU cache, use `LinkedHashMap` with `access-order`.

 **Java**



```
class LRUCache<K, V> extends LinkedHashMap<K, V> {  
  
    private final int capacity;  
  
    public LRUCache(int capacity) {  
        super(capacity, 0.75f, true); // true = access-order  
        this.capacity = capacity;  
    }  
  
    @Override  
    protected boolean removeEldestEntry(  
        Map.Entry<K, V> eldest) {  
        return size() > capacity;  
    }  
}
```

```
}
}
```

Usage:

</> Java



```
LRUCache<Integer, String> cache = new LRUCache<>(3);

cache.put(1, "A");
cache.put(2, "B");
cache.put(3, "C");

cache.get(1);    // 1 becomes recently used
cache.put(4, "D"); // Least recently used entry is removed
```

Internally

HashMap



Fast lookup $O(1)$

Doubly linked list



Maintains access order

Least recently used → head

Most recently used → tail



5-year interview answer:

I can implement a simple LRU cache using LinkedHashMap in access-order mode and override `removeEldestEntry()`. This gives average $O(1)$ lookup/update and automatic eviction of the least recently used entry.

2. How would you design a thread-safe in-memory cache?

For a basic concurrent cache, I would use:

</> Java



```
ConcurrentHashMap<K, V>
```

But if the requirement includes **LRU eviction, TTL, maximum size, statistics, refresh, etc.**, I wouldn't build all of that manually unless required. I'd generally use a mature caching library such as Caffeine.

Basic thread-safe cache

</> Java



```
class Cache<K, V> {  
  
    private final ConcurrentHashMap<K, V> cache =  
        new ConcurrentHashMap<>();  
  
    public V get(K key) {  
        return cache.get(key);  
    }  
  
    public void put(K key, V value) {  
        cache.put(key, value);  
    }  
  
    public V remove(K key) {  
        return cache.remove(key);  
    }  
}
```

ConcurrentHashMap handles concurrent access to individual map operations.

If I need LRU + TTL + maximum size

For a production application, I'd prefer a dedicated caching library rather than manually synchronizing a HashMap .

Conceptually:

In-memory Cache



Concurrent access
Maximum size
Expiration / TTL
Eviction



Statistics

Important design considerations

1. Thread safety
2. Maximum cache size
3. Eviction policy → LRU / size-based
4. TTL / expiration
5. Cache miss handling
6. Concurrent loading / stampede protection
7. Memory limits
8. Metrics and monitoring



★ 5-year interview answer

For a simple thread-safe key-value cache, I'd use `ConcurrentHashMap`. If the cache needs production-grade features such as maximum size, TTL, eviction, statistics, and concurrent loading, I'd use a caching library like Caffeine rather than implementing those mechanisms manually. If I specifically need an LRU implementation for an interview or simple use case, `LinkedHashMap` with access-order can be used, but I'd add appropriate synchronization because `LinkedHashMap` itself is not thread-safe.



ChatGPT can make mistakes. Check important info.